

Implementing BPE

Now we will look at an implementation of the BPE (Byte-Pair Encoding) algorithm. This is not an optimized version, meaning it is not the best version for actually running on a large corpus. The main purpose here is to understand the algorithm well by looking at the code.

1) First, We Need a Corpus

We create a small corpus with a few sentences:

```
corpus = [  
    "This is the Building-LLMs-from-scratch-In-Bangla Course.",  
    "This chapter is about tokenization.",  
    "This section shows several tokenizer algorithms.",  
    "Hopefully, you will be able to understand how they are trained and generate tokens.",  
]
```

What does this mean?

Here, **corpus** is a collection of some texts from which BPE will learn.

2) Next, We Need to Pre-tokenize the Corpus

Since we want to build a BPE tokenizer like GPT-2, the GPT-2 tokenizer is used for pre-tokenization.

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Simply put:

Here, the GPT-2 tokenizer is being loaded so that the text can initially be broken into word-level or basic units.

3) Counting How Many Times Each Word Appears

```
from collections import defaultdict

word_freqs = defaultdict(int)

for text in corpus:
    words_with_offsets = tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str(text)
    new_words = [word for word, offset in words_with_offsets]
    for word in new_words:
        word_freqs[word] += 1

print(word_freqs)
```

What is happening here?

- Each sentence in the corpus is being taken
- It is being pre-tokenized with the GPT-2 tokenizer
- How many times each word/token appears is being counted

Output:

```
defaultdict(int, {
  'This': 3,
  'Ġis': 2,
  'Ġthe': 1,
  ...
})
```

What Is the Ġ Symbol?

In the GPT-2 tokenizer, Ġ usually represents a space.

For example:

- Ġis means " is"
- Ġthe means " the"

That means there was a space before the token.

4) Creating the Base Vocabulary

Now all the characters used in the corpus will be collected to create the base vocabulary.

```
alphabet = []

for word in word_freqs.keys():
    for letter in word:
        if letter not in alphabet:
            alphabet.append(letter)
alphabet.sort()

print(alphabet)
```

Output:

```
[',', '.', 'C', 'F', 'H', 'T', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'k', 'l', 'm', 'n', 'o', 'p', 'r', 's',
 't', 'u', 'v', 'w', 'y', 'z', 'G']
```

What does it mean?

Here, the alphabet has been built using all the unique characters from the corpus.

5) Adding the Special Token to the Vocabulary

The GPT-2 model uses a special token:

```
"<|endoftext|>"
```

So it is being added at the beginning of the vocabulary.

```
vocab = ["<|endoftext|>"] + alphabet.copy()
```

Meaning:

Now the vocabulary will start with the special token, followed by all the characters from the alphabet.

6) Splitting Each Word into Characters

To start BPE training, each word must first be broken down at the character level.

```
splits = {word: [c for c in word] for word in word_freqs.keys()}
```

Example:

If the word is "This", then the split will be:

```
['T', 'h', 'i', 's']
```

7) Function for Calculating Pair Frequency

Now, an important step in BPE training is:

finding which two consecutive characters/tokens appeared the most times.

For that, the function is:

```
def compute_pair_freqs(splits):
    pair_freqs = defaultdict(int)
    for word, freq in word_freqs.items():
        split = splits[word]
        if len(split) == 1:
            continue
        for i in range(len(split) - 1):
            pair = (split[i], split[i + 1])
            pair_freqs[pair] += freq
    return pair_freqs
```

What is happening here?

- The split list of each word is being taken
- Pairs of consecutive tokens are being created
- The more times a word appears in the corpus, the more its pair frequency increases

8) Seeing Some Pair Frequencies

```
pair_freqs = compute_pair_freqs(splits)

for i, key in enumerate(pair_freqs.keys()):
    print(f"{key}: {pair_freqs[key]}")
    if i >= 5:
        break
```

Output:

```
('T', 'h'): 3
('h', 'i'): 3
('i', 's'): 5
('G', 'i'): 2
('G', 't'): 7
('t', 'h'): 3
```

Meaning:

For example:

- the pair (' T ' , ' h ') appeared 3 times
- the pair (' G ' , ' t ') appeared 7 times

9) Finding the Most Frequent Pair

```
best_pair = ""
max_freq = None

for pair, freq in pair_freqs.items():
    if max_freq is None or max_freq < freq:
        best_pair = pair
        max_freq = freq

print(best_pair, max_freq)
```

Output:

```
('G', 't') 7
```

Meaning:

The most frequently used pair is:

('Ĝ', 't')

That is, patterns like " ğ t" appeared the most.

10) Learning the First Merge Rule

So the first merge rule will be:

('Ĝ', 't') -> 'Ĝt'

This is being stored as:

```
merges = {("Ĝ", "t"): "Ĝt"}  
vocab.append("Ĝt")
```

Meaning:

Now the new token " ğt" has been added to the vocabulary.

11) Function for Applying the Merge

Now the new merge rule has to be applied to the splits of all words.

```
def merge_pair(a, b, splits):  
    for word in word_freqs:  
        split = splits[word]  
        if len(split) == 1:  
            continue  
  
        i = 0  
        while i < len(split) - 1:  
            if split[i] == a and split[i + 1] == b:  
                split = split[:i] + [a + b] + split[i + 2 :]  
            else:  
                i += 1
```

```
splits[word] = split
return splits
```

Simply put:

This function does the following:

- looks at the token list of each word
- if **a** and **b** are found side by side
- then it merges them and makes **a+b**

12) Seeing the Result of the First Merge

```
splits = merge_pair("Ġ", "t", splits)
print(splits["Ġtrained"])
```

Output:

```
['Ġt', 'r', 'a', 'i', 'n', 'e', 'd']
```

Meaning:

Before, if "**Ġtrained**" was split, it was:

```
['Ġ', 't', 'r', 'a', 'i', 'n', 'e', 'd']
```

Now **Ġ** and **t** have merged and become:

```
['Ġt', 'r', 'a', 'i', 'n', 'e', 'd']
```

13) Repeatedly Learning Merges with a Loop

Now this process will continue repeatedly until the vocabulary reaches the desired size.

```
vocab_size = 50
```

```
while len(vocab) < vocab_size:
```

```

pair_freqs = compute_pair_freqs(splits)
best_pair = ""
max_freq = None
for pair, freq in pair_freqs.items():
    if max_freq is None or max_freq < freq:
        best_pair = pair
        max_freq = freq
splits = merge_pair(*best_pair, splits)
merges[best_pair] = best_pair[0] + best_pair[1]
vocab.append(best_pair[0] + best_pair[1])

```

What is the loop doing here?

Each time it:

- calculates pair frequency
- finds the most frequent pair
- merges the pair
- saves the merge rule
- adds the new token to the vocabulary

14) How Many Merge Rules Were Learned?

```
print(merges)
```

Output:

```

{
('Ĝ', 't'): 'Ĝt',
('i', 's'): 'is',
('e', 'r'): 'er',
('Ĝ', 'a'): 'Ĝa',
('Ĝt', 'o'): 'Ĝto',
('e', 'n'): 'en',
('T', 'h'): 'Th',
('Th', 'is'): 'This',
...
}

```


Meaning:

Here, the model has learned which pairs should be merged to create new tokens.

For example:

- ('i', 's') -> 'is'
- ('T', 'h') -> 'Th'
- ('Th', 'is') -> 'This'

In this way, larger tokens have gradually been created.

15) What Does the Final Vocabulary Look Like?

```
print(vocab)
```

Output:

```
[ '<|endoftext|>', ',', '.', 'C', 'F', 'H', 'T', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'k', 'l', 'm', 'n', 'o',  
'p', 'r', 's', 't', 'u', 'v', 'w', 'y', 'z', 'Ġ', 'Ġt', 'is', 'er', 'Ġa', 'Ġto', 'en', 'Th', 'This', 'ou', 'se',  
'Ġtok', 'Ġtoken', 'nd', 'Ġis', 'Ġth', 'Ġthe', 'in', 'Ġab', 'Ġtokeni']
```

Meaning:

The final vocabulary is made up of:

- the special token
- the initial alphabet
- all the learned merged tokens

16) Important Note

If `train_new_from_iterator()` is used, the exact same vocabulary may not be produced.

Why?

Because if multiple pairs have the same frequency:

- our code takes the first one it finds
- but the Hugging Face Tokenizers library chooses the pair based on internal IDs

That means even if the algorithm is the same, implementation details can differ.

17) Tokenizing New Text

Now new text can be tokenized using the learned merge rules.

```
def tokenize(text):
    pre_tokenize_result = tokenizer._tokenizer.pre_tokenizer.pre_tokenize_str(text)
    pre_tokenized_text = [word for word, offset in pre_tokenize_result]
    splits = [[l for l in word] for word in pre_tokenized_text]
    for pair, merge in merges.items():
        for idx, split in enumerate(splits):
            i = 0
            while i < len(split) - 1:
                if split[i] == pair[0] and split[i + 1] == pair[1]:
                    split = split[:i] + [merge] + split[i + 2 :]
                else:
                    i += 1
            splits[idx] = split

    return sum(splits, [])
```

What does this function do?

When new text comes:

- it pre-tokenizes first
- it splits each word into characters
- it applies the learned merge rules one by one
- finally, it returns the final token list

18) Example: Tokenizing

```
tokenize("This is not a token.")
```

Output:

```
['This', 'Ġis', 'Ġ', 'n', 'o', 't', 'Ġa', 'Ġtoken', '.']
```

Explanation:

Here some parts have merged and become larger tokens:

- This
- Ġis
- Ġa
- Ġtoken

But `not` did not fully merge, so it got split into:

- Ġ
- n
- o
- t

19) Problem with Unknown Characters

It is said in the text:

our implementation will give an error if an unknown character appears.

Why?

Because we did not keep all possible characters or bytes in the initial vocabulary.

In GPT-2, this problem does not happen, because GPT-2 uses:

byte-level BPE

That means all possible bytes are kept in the initial vocabulary, so there is no chance of creating an unknown character.

20) The Whole Topic in Very Simple Language

In this implementation, the BPE algorithm works step by step like this:

Training phase:

- the corpus is taken
- it is pre-tokenized
- the frequency of each word is calculated
- a base vocabulary is built using all unique characters

- the words are split into characters
- the most frequent pair is found and merged
- the new token is added to the vocabulary
- this is repeated again and again

Tokenization phase:

- the new text is pre-tokenized
- it is split into characters
- the merge rules learned during training are applied
- the final tokens are created